

Project Report

DD2482

Automated Software Testing and DevOps

Authors

Felix Hansson Huber (felixhh@kth.se)

Pierre Segerström (pise@kth.se)

The architecture and processes

This project utilizes an existing web application, and integrates a CI/CD DevOps pipeline “on top” of it. The pipeline is invoked on each push to the repository, and when pushes are made to the `main` branch (solely through pull requests), the deployment is also run.

On each push, the following happens, in order:

CI – On GitHub-hosted runners:

- **Verifications:**
Runs test suites for the frontend and backend workspaces. We utilized the initial project’s existing test suite, and then set test coverage thresholds (statement, branch, and function) based on a test coverage report as a baseline. In the same way, we also built a test suite for the backend with test coverage thresholds. Besides testing, we’re also running qualitative aspects like linting and formatting, and finally package security auditing.
- **Build and Push:**
Builds the Docker images that are needed to run the full application infrastructure (frontend + backend). These are then pushed to Docker Hub with tags for the branch and commit hash. Builds which result from `main` also receive the “latest” tag, making the origin of each deployed image easier to trace.
- **Image builds from Docker Hub are tested:**
The images are then pulled from Docker Hub and tested as part of a temporary version of the final infrastructure. Terraform checks that the expected resources are created, and that both the frontend and the backend health endpoints can be reached through nginx.

CD – On self-hosted runner:

The final part of the pipeline happens only on merges to the `main` branch (through PRs).

- **Deployment to self-hosted runner:**
The self-hosted runner pulls the images from Docker Hub, where Terraform will then handle what needs to be updated on the live infrastructure through its built-in state management.

The deployed live infrastructure

We expanded upon the initial project, by adding a login feature, as well as a “Watchlist” feature. This was added in order to get an infrastructure that more closely resembles the dependencies and services of a web app in production.

When a user opens the application, Nginx directs requests to the frontend containers or to the backend API. The frontend retrieves show information directly from an external API ([TVMaze](#)). For login and watchlist actions, it calls our backend, which stores users, sessions and watchlist show IDs in Postgres.

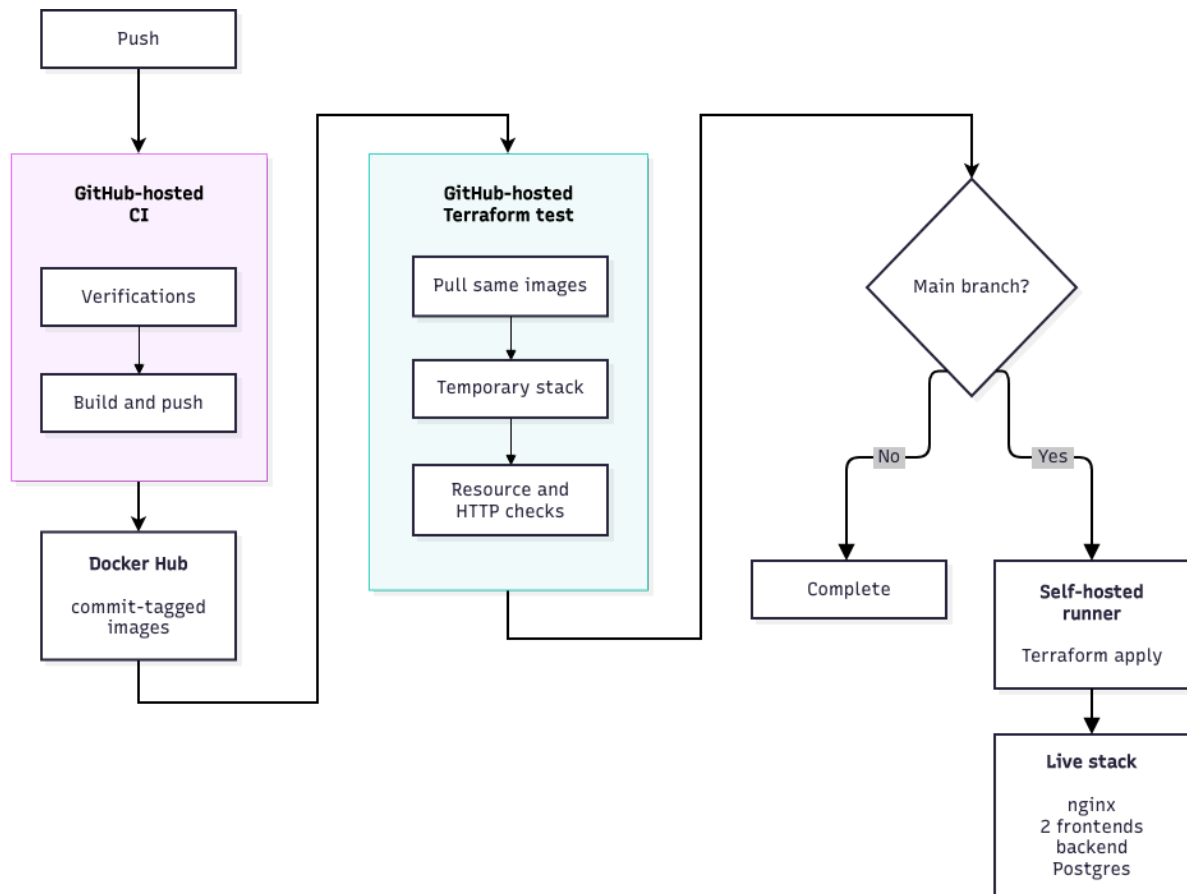


Figure 1: Diagram illustrating the flow of the DevOps pipeline

Justification of key design decisions

Version control platform:

We chose GitHub due to the “plug-and-play” nature of developing a DevOps pipeline on a well-supported platform.

Self-hosted runners:

To support learning further, we wanted to own the pipeline end-to-end, rather than relying on a cloud application host service. This gave us the opportunity to monitor the deployment more closely in the final part of the pipeline. Furthermore, having this “local approach” is convenient for the sake of sharing this repo as a project hand-in with reduced external dependencies.

Choice of project to build the pipeline on:

We chose “[MazeFlix](#)” as a base since we wanted the experience to perform this exercise on a project none of us had seen before. We chose a Wweb-based TypeScript SPA application because it would offer plenty of testing and qualitative CI gates out of the box, so that we could focus more on the pipeline itself. It was a good learning exercise to first assess the state of the existing project, as a foundation to shape a DevOps pipeline. We were fortunate that the project already had a test suite. This allowed us to introduce regression testing and coverage thresholds as CI gates right away. Otherwise, establishing a test suite would have been our first priority.

Terraform:

Since we have a network of Docker containers, we could just as well have used “Docker Compose”. This was an intentional decision from us, since the “local approach” wasn’t the end-goal, but rather a way to monitor the pipeline more closely as previously mentioned. By utilizing Terraform instead, we can model how this scenario could interact with “real” cloud-based providers, while running it locally.

To facilitate easy clean-up of the deployment, we added a manually triggered “destroy workflow”. This runs `terraform destroy` and uses the same state as the original “terraform-apply” job does. To ensure no accidental or unauthorized destroys are fulfilled, an approval from another contributor is needed. This is achieved by specifying an environment with pre-set rules in the destroy workflow. One could assume that the same environment should be used in the terraform-apply job. However, as the environment demands an approval from another contributor, we opted to not specify it in that job as we want the deployment to happen automatically once a PR is merged.

Limitations and trade-offs

Deploying to local runner:

While the local, self-hosted deployment gives us more control and visibility for learning, it also requires a machine with Docker and a configured Github Actions runner to remain available. This adds further manual steps and maintenance for our DevOps pipeline. The workflow is also currently restrained to Unix-systems, which reduces portability.

Lack of redundancy:

Since all services, database storage and the deployment state lives in one machine, this also makes it a single point of failure.

Missing availability guarantees:

Deployment currently lacks automatic rollback or a strategy guaranteeing uninterrupted service during updates. These mechanisms could be implemented on our local machine, but would require additional configuration and deployment logic. Utilizing a managed hosting platform with built-in support for these aspects could simplify this work.

Terraform introduces complexity:

Like previously mentioned, using Terraform instead of Docker Compose adds unnecessary complexity for such a small Docker network. Additionally, moving to a cloud-based provider would require changes to the resource definitions.

Use of AI tools

AI has been used during exploration, brainstorming, and task delegation throughout the separate parts of building this project.

Planning:

When we initiated this project, we set a technical implementation roadmap based on our [Project proposal](#). AI was used to explore blind spots and evaluate trade-offs within our already-set plan.

Development & Review:

For each section's development, we delegated a technical specification for the task at hand (each separate GitHub issue) to Claude or Codex, which laid the foundation for the implementation. This then went through collaborative reviews by both of us, where we concluded modifications or changes that were needed to fit our plan and use case. This was the process of each pull request, which included: workflow development, extensions to the application's features, and the Terraform configuration.